Pano Klironomos

CSCI 580

Technical Report: SA vs GA

Problem Context

The traveling salesman problem tasks us to find the shortest path to visit every city in a set of cities exactly once and return to the start. Testing every possible combination of cities would be far too computationally expensive, so we compared two different solutions to the problem: Simulated annealing and Genetic algorithm. Our setup has 40 randomized cities and we used nearest neighbor as our baseline tour of the cities to contrast against our best SA and GA tours.

Implementation

Simulated Annealing works by repeatedly making small adjustments to get a shorter path. Our implementation starts with the nearest neighbor and then uses 2-opt which randomly selects two cities and reverses the section between them. If the new tour ends up shorter, we accept it, and if it is not shorter, we use the current temperature to determine if we still accept the longer tour:

```
        if delta <= 0:
          accept = True
        else:
          accept = rng.random() < math.exp(-delta / T)
```

Higher temperatures allow SA to explore different solutions and escape local minima, so SA is less likely to accept longer tours as the temperature decreases. We cool the temperature down as we iterate on the tour, allowing SA to explore early and then settle later on:

```
        T = T * cfg.alpha
        T = max(T, 1e-12)
```

Genetic Algorithm uses generations of many tours rather than making tweaks to one tour. My implementation uses a population of 200 tours over 400 generations. I set the mutation rate to 0.5 which increased my improvement substantially. Genetic Algorithm also starts with our nearest neighbor tour, and then uses these three methods to evolve its population:

Tournament selection chooses the shortest tour out of a random sample of tours to reproduce:

```
 sampleIndices = rng.sample(range(len(pop)), k)
winner = sampleIndices[0]
for i in range(k):
  j = sampleIndices[i]
  if lengths[j] < lengths[winner]:
    winner = j
return pop[winner]
```

Order crossover combines two parent tours by using a random slice of the first parent's tour and the remaining cities in the second parent's tour, essentially "reproducing", while making sure every city only appears once in the child tour:

```
   # put parent 1 slice into child (crossover)
  for i in range((cut2+1)-cut1):
    ind = cut1 + i
    child[ind] = parent1[ind]
  c = cut2 + 1

  # read through parent 2 after cut 2 and
  # fill the rest of child after cut2 wrapping around, with that, skipping duplicates
  for i in range(tourlen):
    j = cut2 + 1 + i
```

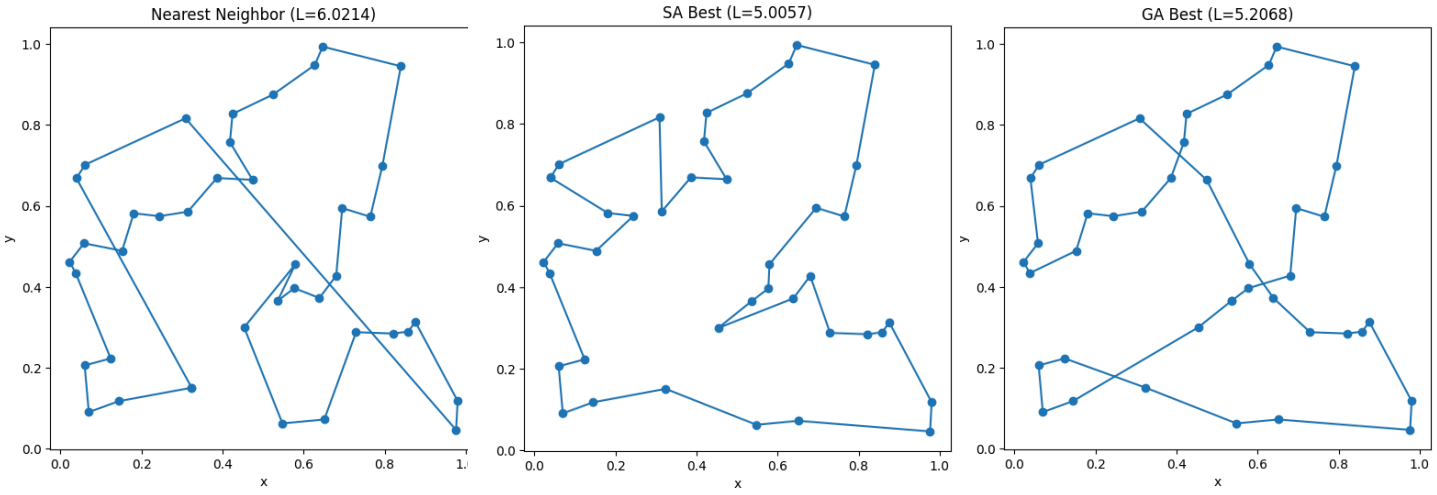
```
    j %= tourlen
    c %= tourlen

    if parent2[j] not in child:
      if child[c] != -1:
        while child[c] != -1:
          c+=1
          c %= tourlen
      child[c] = parent2[j]
      c+=1
      c %= tourlen
```

Mutate swap swaps two cities in a tour at random, which adds variation in the population and allows for beneficial mutations to reproduce:

```
  allIndices = list(range(len(tour)))
  ind1 = allIndices[rng.randrange(0,len(tour))]
  allIndices.remove(ind1)
  ind2 = allIndices[rng.randrange(0,len(tour)-1)]

  tour = tour.copy()
  tour[ind1], tour[ind2] = tour[ind2], tour[ind1]
  return tour
```



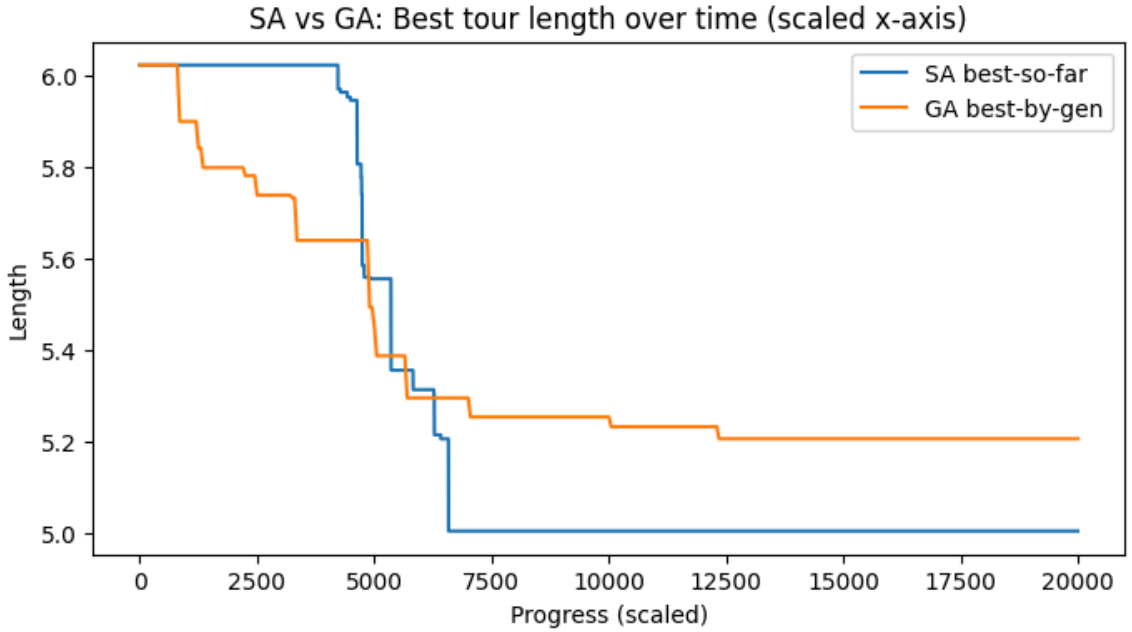
Setup and Results

I did not change SA's parameters; It used a t0 of 0.2 and an alpha of 0.9995. The best length I obtained with SA was 5.0057, which was 16.87% improvement on our nearest neighbor baseline. The best tour produced by SA has no overlaps and looks fairly optimal.

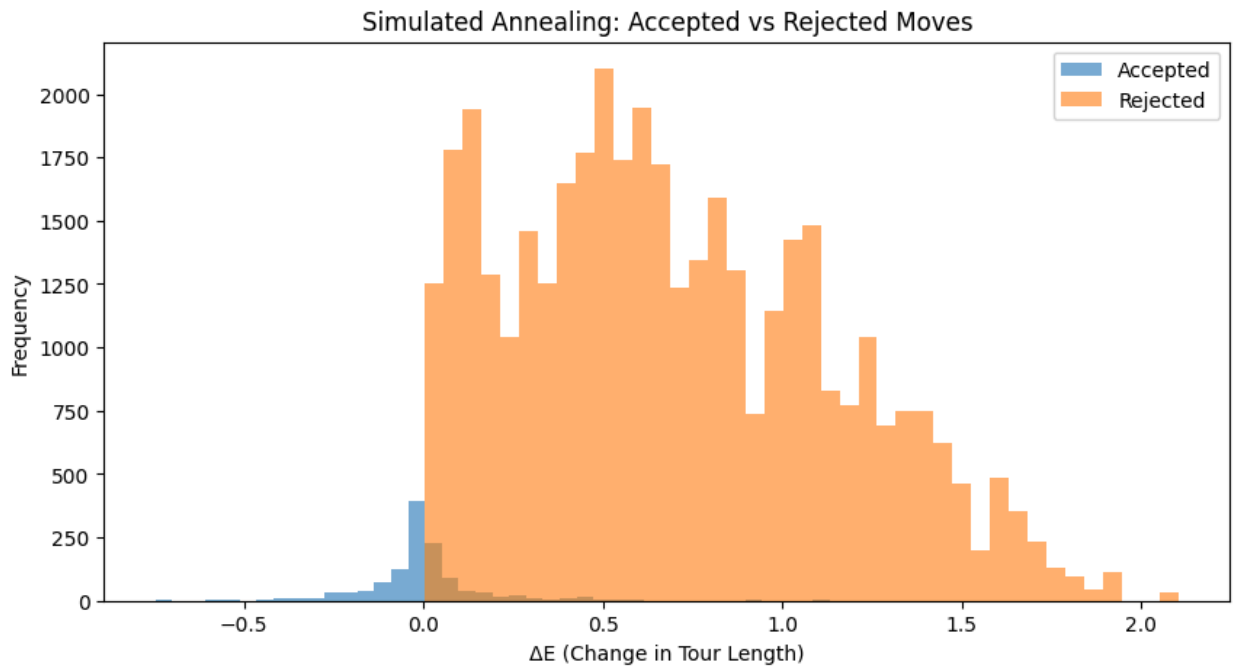
For GA, I changed the mutation rate from 0.2 to 0.5, and tournament size from 5 to 3. The mutation rate increase allowed GA to try new solutions more often, and the decrease in tournament size allowed the longer tours to have a better chance at reproducing which preserved our population diversity. Changing these two config variables for GA increased my improvement by over 10% alone. The best length I obtained with GA was 5.2068, which was a 13.53% improvement on our nearest neighbor baseline. GA's best tour was an improvement from nearest neighbor but is sub-optimal and has several overlaps.

Simulated Annealing performed better than Genetic Algorithm for this experiment, achieving a shorter, and overall cleaner tour. This is possibly because SA's 2-opt segment reversal preserved optimal parts of the path better than GA's mutate swap did. GA's random swap mutations seemed to lead to a more jumbled tour.

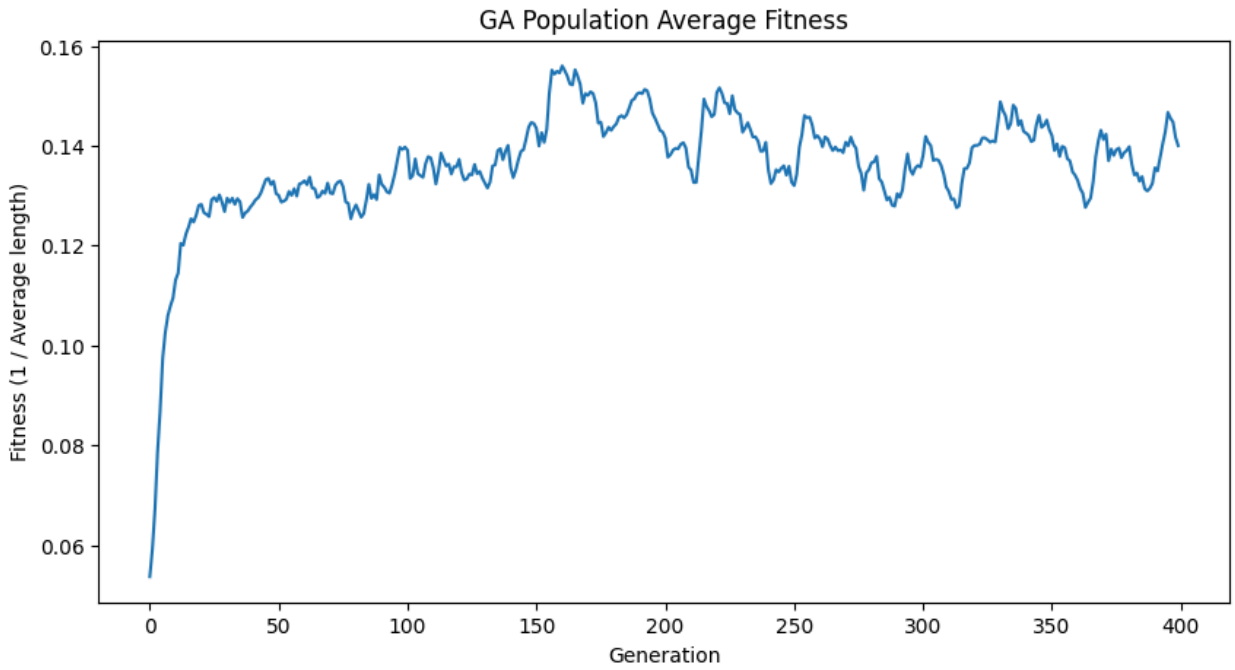
Analysis



We can see by the graph that GA converged earlier on, but SA converged very quickly once it began to lower the tour length. SA ultimately found a shorter tour than GA, and both converged to their final tour solution at around the same point in this scaled progress graph. GA showed more variability from its configuration settings. The variables for GA drastically changed the results, with small changes to the mutation variable causing large differences in success. SA took longer to begin converging but found a significantly better tour in the end.



The histogram of accepted vs rejected delta for SA showed that it occasionally accepted positive delta values, so we were likely able to escape local minima. This is because of our temperature value being initially high and then cooling down. By accepting longer tours with higher temperatures, SA was able to look for bigger-picture solutions and not get stuck in sub-optimal solutions.



Our population fitness vs generation graph shows us that GA likely lost most of its genetic diversity once its shortest tour length plateaued around generation 20. Tournament selection favored the best solutions, so the dominant genes likely stayed after the algorithm settled. The best tour length plateaued at a higher length than SA, meaning our GA population likely was not diverse enough toward the end to find a better solution. GA gains and loses some fitness over the rest of the generations after its plateau, but never significantly improves its solution. We can tell by this premature plateau and the suboptimal crossed-over GA best tour visualization, that our algorithm or configuration was not enough to beat SA for the traveling salesman problem.